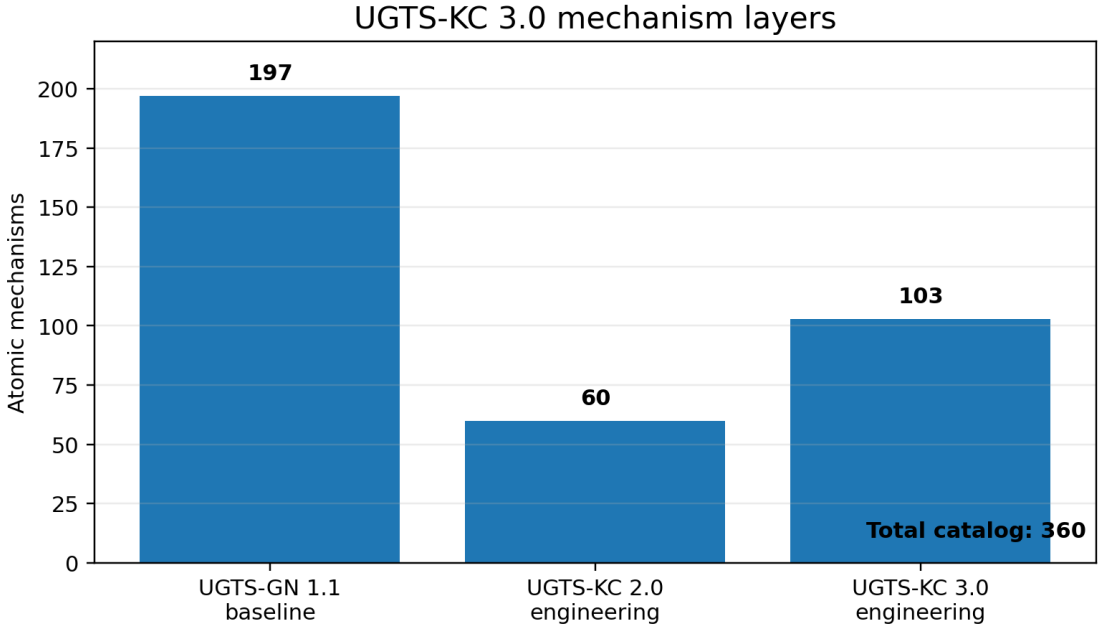
UNIFIED GEOMETRIC-TOPOLOGICAL SUBSTRATE

UGTS-KC 3.0

Jet Kinematics, Constraint and Contact Calculus, Certified Hybrid Events, Persistent Topology, Multiscale Patterns, Geometric Dynamics and Reproducible Runtime Contracts

Prepared for **Tom Klootwijk** | Release date: **16 August 2026**

360	103	182
atomic mechanisms	new 3.0 mechanisms	passing tests



Catalog composition: source-normalized baseline plus append-only 2.0 and 3.0 engineering layers.

READING RULE

The authoritative core is a directly queryable state-and-event substrate. Projection, GPU execution and physical devices remain optional consumers. The canonical chain is support -> compatibility -> guard/certification -> verified event -> atomic transition -> lineage and novelty log.

Requester attribution was supplied by the requester and was not independently verified. Full identity metadata is isolated in REQUESTER_ATTRIBUTION.txt.

Contents

EVIDENCE KEY

M001-M197 are source-derived or normalized. M198-M257 are the carried-forward 2.0 engineering layer. M258-M360 are new 3.0 engineering mechanisms. Validation labels distinguish implemented, tested, prototype and specified contracts.

Executive synthesis

The supplied reports already establish the mature UGTS architecture. The original report defines a finite grammar over typed state, explicit relation surfaces, local radial-angular support, compatibility, earliest-event solving, transition routing, lineage and an external novelty log. The GPU-native addendum carries that architecture into a 197-mechanism normalized baseline and a direct compute/hardware boundary. [SR1 pp. 1-2, 13-16; SR2 pp. 2-7]

UGTS-KC 3.0 does not replace that core. It expands the calculational layer that sits between typed state and verified event commitment. The upgrade makes higher-order motion explicit, adds constrained and unilateral contact dynamics, classifies degenerate and simultaneous events, introduces bounded certification utilities, promotes topology to measurable filtrations and invariants, adds multiscale pattern generators and spatial indices, supplies geometric and field-dynamics integrators, and makes uncertainty and replay integrity first-class.

The complete catalog now contains 360 atomic mechanisms:

- M001-M197: source-derived or source-normalized UGTS-GN 1.1 baseline.
- M198-M257: UGTS-KC 2.0 engineering expansion for patterns, implicit fields, topology calculus, kinematic calculus and bounded dynamics.
- M258-M360: 103 new UGTS-KC 3.0 engineering mechanisms.

The new mechanisms are explicitly engineering-derived. They are not presented as quotations or discoveries from the supplied PDFs. Their admission test is the same as the source reports' admission rule: each item must be representable as a typed state field, transform, support predicate, relation, compatibility gate, event policy, transition, invariant, lineage rule, measurable dynamics operator or reproducibility contract.

Decision

GO for a bounded 3.0 reference substrate. The release is suitable as a mathematical specification, exchange schema, executable reference, test corpus and implementation roadmap. It is not evidence of universal constant-time event solving, zero storage or energy, physical-GPU speed, arbitrary-field exact roots, optical advantage, physical non-orientable self-assembly or complete state in one bit.

1. Source basis and evidence boundary

1.1 Supplied reports

SR1 - Unified Geometric-Topological Substrate. This report defines the UGTS-0 authority split: finite grammar and typed state, relations and guards, topology and identity, external novelty, support admission, compatibility, event solving and transition routing, with game, graphics and hardware as adapters. It states that projection is optional and that one-bit values are schema-bound flags rather than complete state. [SR1 pp. 1-2]

SR2 - Unified Geometric-Topological Substrate GPU-native Addendum. This report retains the canonical query-first sequence and expands the normalized catalog to 197 mechanisms. It adds direct Vulkan/SPIR-V execution, typed state/event ABIs, measured software-device performance, compression contracts and bounded physical-device mappings. It explicitly separates source-derived operators, engineering normalization, measured package evidence and unsupported claims. [SR2 pp. 2-4]

The two PDFs are not redistributed. Their SHA-256 hashes and privacy-safe descriptions are included under `sources/`.

1.2 Three evidence layers

Layer	IDs	Meaning
Source-normalized baseline	M001-M197	Extracted or normalized from the supplied source reports and their internal source register.
UGTS-KC 2.0 engineering	M198-M257	Pattern, field, topology, kinematic and bounded-dynamics additions carried forward into 3.0.
UGTS-KC 3.0 engineering	M258-M360	New higher-order kinematics, contact, certified events, persistence, multiscale dynamics, uncertainty and runtime contracts.

Every catalog row carries a layer, disposition, validation label and implementation reference. `IMPLEMENTED + TESTED` means exercised by the bundled reference tests. `SPECIFIED` means the contract is defined but a complete production implementation is not claimed.

1.3 Non-negotiable boundaries

- Coordinates are not identity. Stable identity requires address, ancestry, invariants and ordered novelty.
- Co-location is not coupling. Compatibility remains a separate typed decision.
- A field zero is a guard or relation surface; it does not imply an exact signed distance or an exact root for every trajectory.
- A one-bit field is a route, mask, orientation, latch or validity flag under a schema; it is not the complete state.
- Topological names are accepted only through explicit gluing, incidence, filtration, group or transfer data.
- A memory or performance reduction is not a correctness proof.
- Hardware claims require named devices, calibrated error, full-system energy and comparison with a conventional baseline.

2. Canonical architecture and the 3.0 extension

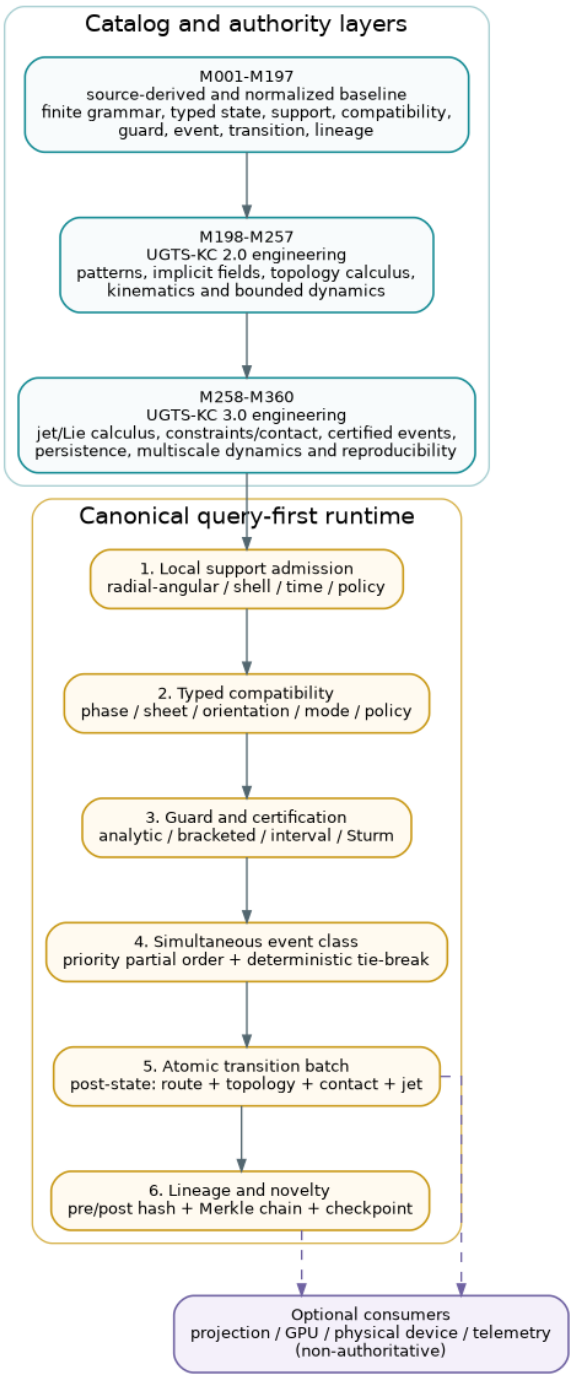


Figure 2. UGTS-KC 3.0 preserves source authority while adding certified event, contact, topology and replay layers.

The canonical shorthand remains:

```
local support -> typed compatibility -> guard/certification
-> simultaneous event class -> atomic transition batch
-> route/topology/contact/jet update -> lineage + novelty log
```

UGTS-KC 3.0 inserts additional structure without changing authority:

1. **Jet state** exposes derivatives through snap rather than hiding them inside an integrator.
2. **Constraint and contact layers** turn manifold, gap, impact and friction semantics into typed operators.
3. **Certified event logic** returns crossing direction, tangency/grazing status, interval enclosures and root-count information where supported.

4. **Simultaneous-event batching** groups events inside a declared tolerance, applies a priority partial order and commits atomically.
5. **Topological observables** use chain complexes, filtrations, barcodes, monodromy and explicit invariants rather than visual analogy.
6. **Uncertainty and replay** attach error budgets, canonical hashes, Merkle event chains and checkpoints.
7. **Capability routing** makes backend support and fallbacks explicit.

The architecture remains query-first. A frame loop may schedule work; it is not the state authority. A GPU kernel may accelerate a declared operator; it does not redefine the world schema. A physical sensor may produce a measured guard; its calibration and uncertainty become state and event metadata.

3. Mechanism catalog and versioning

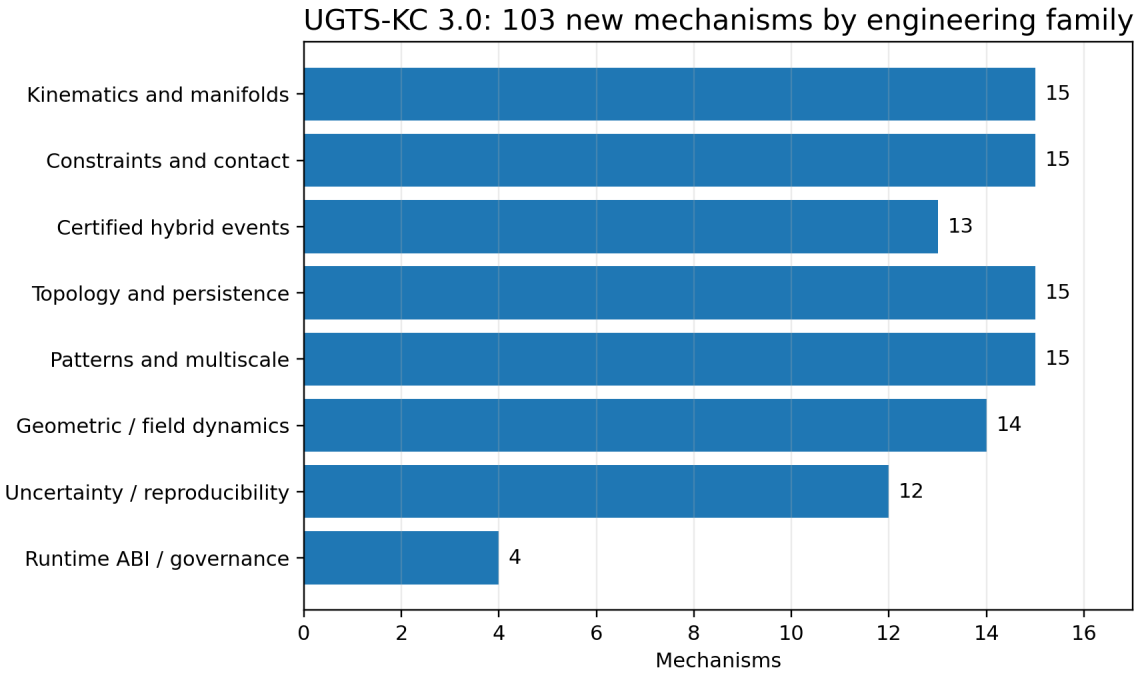


Figure 3. Distribution of the 103 new 3.0 mechanisms across eight engineering families.

3.1 Catalog growth

The 3.0 catalog is append-only. Existing identifiers are preserved, so traces and claims ledgers remain referentially stable. The new range M258-M360 adds 103 mechanisms grouped into eight engineering families.

Family	Count	Main range
Kinematics and manifolds	15	M258-M272
Constraints and contact	15	M273-M287
Certified hybrid events	13	M288-M300
Topology and persistence	15	M301-M315
Patterns and multiscale structure	15	M316-M330
Geometric and field dynamics	14	M331-M344
Uncertainty and reproducibility	12	M345-M356
Runtime ABI and governance	4	M357-M360

3.2 Version rule

A mechanism identifier names one atomic operator or contract. Backward-compatible clarification may improve its description, but a semantic replacement receives a new identifier. World files state their schema version and capability requirements. Backends publish a capability manifest and a fallback map; silent substitution is not allowed.

3.3 Admission and kill rules

A proposed mechanism enters the core only when its inputs, outputs, units, frame, tolerances and failure states are explicit. It is rejected or demoted when it relies on an undefined physical analogy, an absolute performance statement, hidden state, unbounded recursion, an unmeasurable guard or an implementation that does more work than the materialization it claims to avoid.

4. Jet, differential and Lie kinematics

4.1 Jet-state tower

The 3.0 reference state extends motion to

```
J(q,t) = (x, v, a, j, s; frame, length_unit, time_unit)
```

where `x` is position, `v` velocity, `a` acceleration, `j` jerk and `s` snap. The orders are stored separately, dimension-checked and evaluated through a declared truncation. Zero-valued higher orders are explicit rather than absent.

This makes event prediction, limit checking and trajectory handoff inspectable. It also prevents an overloaded symbol from standing simultaneously for time, torque, transformation and phase - a correction already emphasized by the earlier UGTS synthesis.

4.2 Material and covariant derivatives

For a scalar field `f(x,t)` and trajectory velocity `v`, the reference material derivative is

```
Df/Dt = partial(f)/partial(t) + grad(f) dot v
```

A covariant derivative is a separate contract. Ordinary component differentiation is not treated as coordinate invariant when a chart connection or metric is required.

4.3 Lie flow and rigid motion

The release implements or specifies:

- Lie brackets for noncommuting local flows.
- SE(3) adjoint transfer of twists and wrenches between frames.
- Stable small-angle SO(3) left Jacobian.
- Bounded SE(3) logarithm under a declared rotation branch.
- Dual-quaternion rigid transforms and normalized blending.
- Geodesic interpolation and local retraction updates.
- Quaternion Lie-group stepping for orientation.

The practical rule is that rigid motion is composed on the group, not by unconstrained component drift. Every transform carries a frame convention.

4.4 Curve invariants and transported frames

Curvature and torsion are computed from derivatives when the required rank and speed conditions hold. Bishop frames are retained for near-zero curvature where Frenet frames are unstable. Closed transport may accumulate holonomy; this is recorded as a frame-state result, not interpreted as free physical rotation.

4.5 Timing and synthesis

Limit-aware time scaling separates path geometry from timing and applies velocity, acceleration and jerk bounds. Differential flatness is included as a reconstruction contract for systems that actually expose flat outputs; it is not assumed for arbitrary dynamics.

5. Constraint and contact calculus

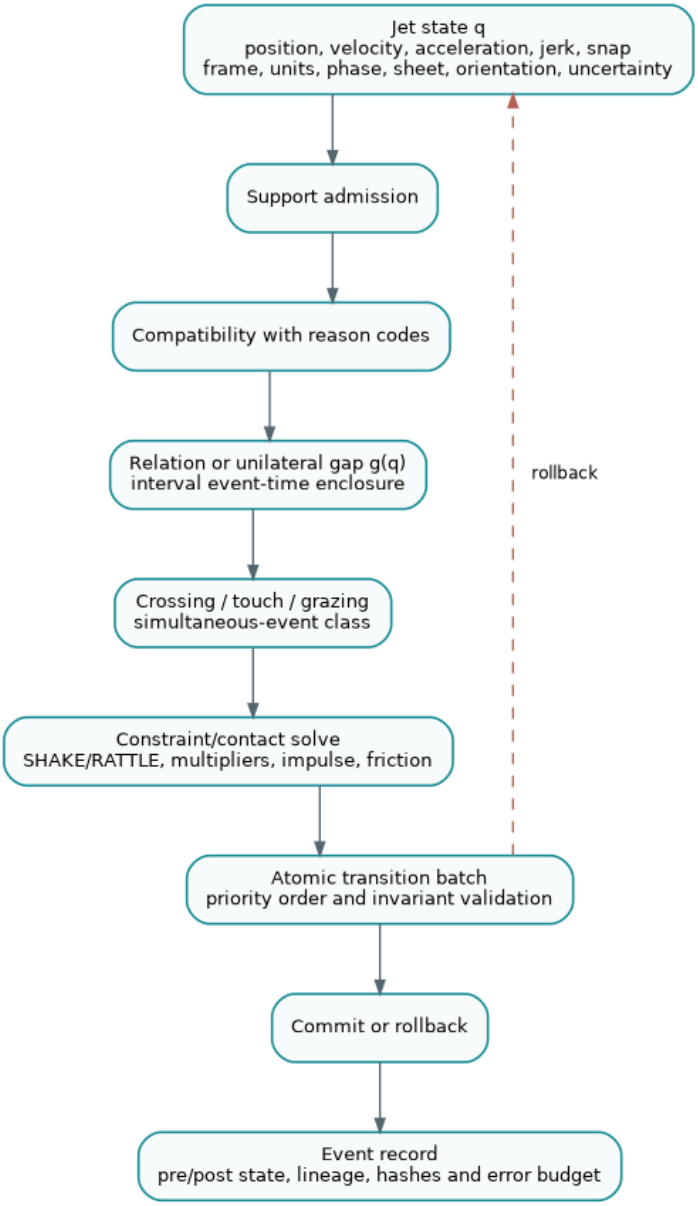


Figure 4. Contact and hybrid-event path from jet state through certified event classification to atomic commit or rollback.

5.1 Constraint types

A holonomic constraint is represented as `phi(q,t)=0`; a nonholonomic constraint uses a Pfaffian form `A(q,t) v = b(q,t)`. Constraint functions, Jacobians, tolerance, stabilization method and frame are declared in the schema.

The reference package includes:

- constraint Jacobian evaluation,
- Baumgarte stabilization,
- SHAKE position projection,
- RATTLE velocity projection,
- small dense Lagrange-multiplier solves,
- null-space constrained acceleration,
- projected symplectic circle integration.

Projection residuals are returned. A projected point without its residual and tolerance is not a verified constrained state.

5.2 Unilateral contact

A contact candidate uses a signed gap `g(q)`. The ideal normal complementarity condition is

```
g >= 0, lambda_n >= 0, g * lambda_n = 0
```

with finite tolerances in software. A negative gap is penetration, zero is contact within policy and positive is separation. The guard direction distinguishes entering, leaving and touching.

5.3 Impact and restitution

The normal impulse solves a declared effective-mass relation and applies a coefficient of restitution in `[0,1]`. Restitution is not used to manufacture energy. Near-resting contacts may switch to a non-bouncing policy to avoid chatter.

5.4 Friction

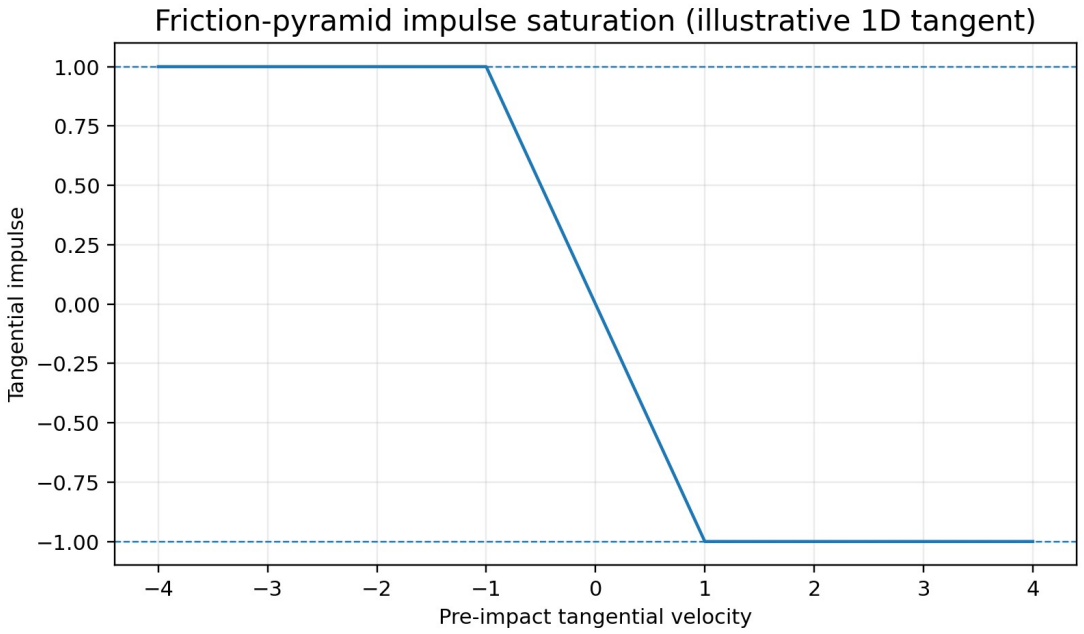


Figure 5. Illustrative friction-pyramid saturation for one tangent direction.

Coulomb friction is a cone `||J_t|| <= mu J_n`. The reference includes a friction-pyramid approximation and bounded tangent impulse. The approximation resolution, warm-start state and solver tolerance must be visible because they can change event order and energy dissipation.

5.5 Contact manifold and warm start

Multiple nearby contact points are reduced to a bounded manifold using separation and normal criteria. Cached impulses are keyed to stable contact identifiers and invalidated when topology, frame or feature identity changes. Warm starting is an optimization, not authoritative state unless included in deterministic replay.

6. Certified hybrid event calculus

6.1 Event classification

A guard result is not reduced immediately to true/false. The 3.0 event classifier distinguishes:

- rising crossing,
- falling crossing,
- touch/tangency,
- grazing candidate,
- coincident or unresolved interval,
- no root in the certified interval.

Derivative information and interval residuals accompany the classification. Tangency is not silently reported as a crossing.

6.2 Simultaneous events

Events whose time enclosures overlap within `simultaneous_tolerance` form an equivalence class. The class is processed through:

1. a declared priority partial order,
2. a deterministic tie-break key for otherwise incomparable events,
3. a proposed state-patch batch,
4. invariant and compatibility validation,
5. atomic commit or rollback.

This prevents result dependence on thread scheduling, hash-table order or small floating-point perturbations.

6.3 Certification utilities

The package includes bounded reference utilities for:

- interval event enclosure,
- Lipschitz root exclusion,
- interval Newton contraction,
- Sturm polynomial root counting,
- interval scalar/vector bounds.

These tools certify only their declared assumptions. They do not make arbitrary implicit fields globally solvable.

6.4 Zeno and hysteresis policy

A Zeno detector watches shrinking inter-event times and reports accumulation rather than iterating forever. Dwell timers, hysteresis and debounce remain explicit policy. A production system must choose whether to regularize, aggregate, stop or hand off the accumulation.

7. Persistent topology and topological events

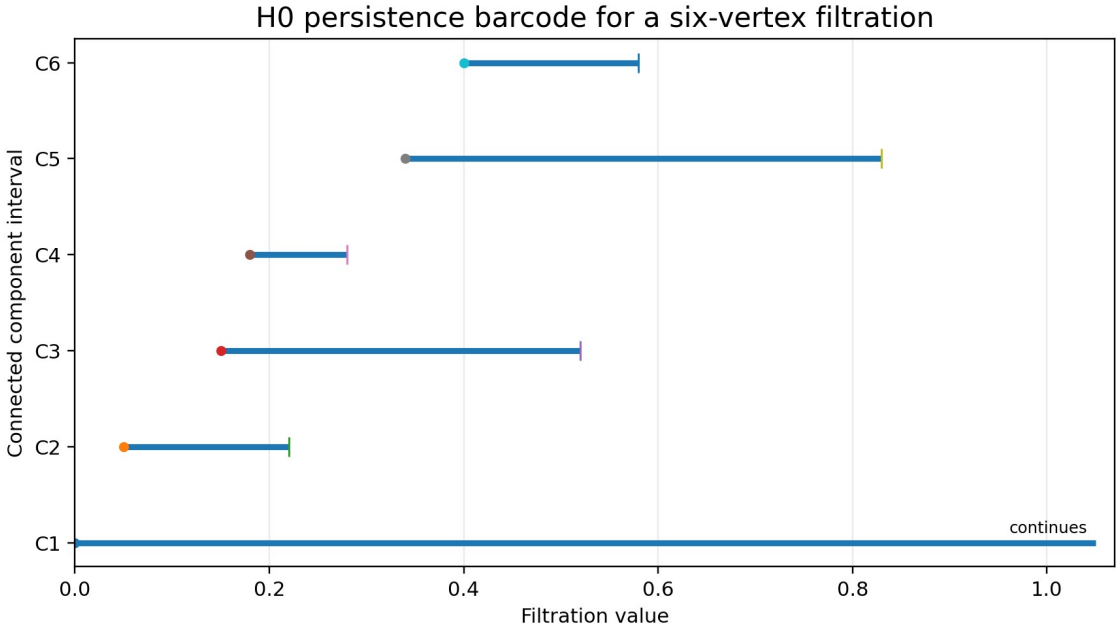


Figure 6. Bounded H0 persistence example. Infinite continuation is a declared interval status, not an infinite computation.

7.1 From names to combinatorics

The earlier UGTS reports already require explicit quotient and gluing maps for Mobius and Klein motifs. Version 3.0 generalizes that discipline: topology is carried by oriented incidence, chain maps, filtrations, group presentations, monodromy or measured invariants - not by a suggestive rendering.

7.2 Simplicial and cubical structures

The reference package supports oriented simplex incidence and checks that boundary composed with boundary is zero. It constructs small Vietoris-Rips complexes, an alpha-complex proxy and cubical lower-star orderings. These are bounded educational implementations, not replacements for specialized large-scale topology libraries.

7.3 Homology and persistence

The package includes:

- Betti-vector calculation for bounded complexes,
- union-find H0 persistence,
- persistence intervals and barcodes,
- a bounded diagram-distance approximation,
- topology-threshold event generation.

A persistent feature becomes an event only under a declared minimum lifetime, dimension and confidence policy. Short-lived features can be retained as uncertainty rather than treated as structure.

7.4 Differential and algebraic topology contracts

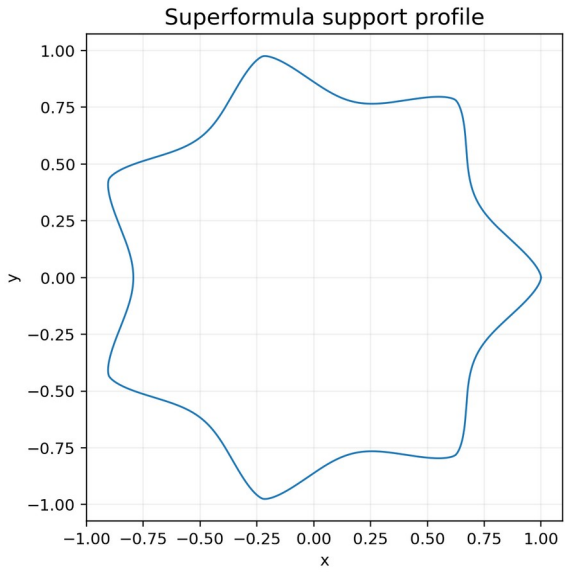
A graph Hodge Laplacian supplies a discrete exterior-calculus foothold. Harmonic representatives, winding/linking estimates, fundamental-group presentations and covering-space monodromy are included as explicit contracts. Each requires valid discretization and orientation data.

7.5 Topological change events

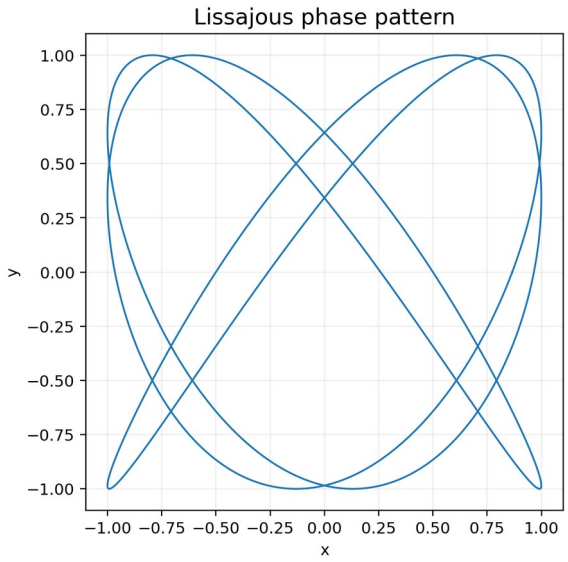
Birth, death, merge, split, handle or monodromy changes may be emitted as typed events. The event stores the filtration parameter, before/after invariant, representative or feature identifier, confidence and lineage. Visual change alone is not authoritative.

8. Pattern, field and multiscale expansion

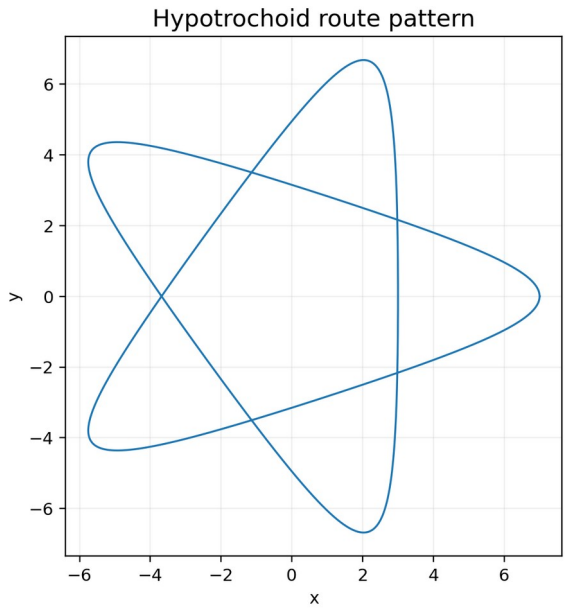
8.1 Carried-forward 2.0 patterns



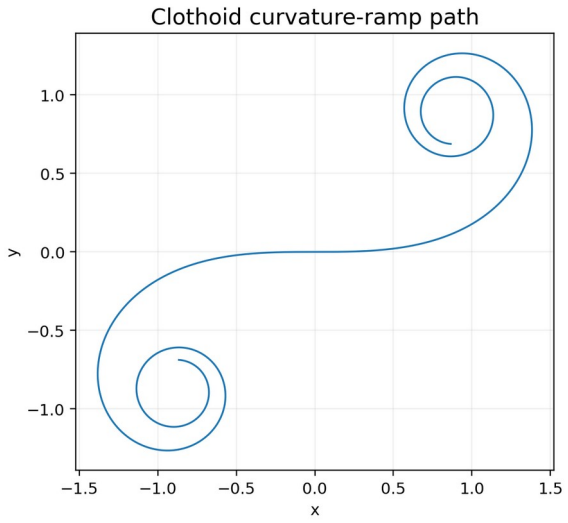
Superformula support profile



Lissajous phase pattern



Hypotrochoid route pattern



Clothoid curvature-ramp path

Figure 7. Carried-forward parametric pattern examples. Each curve is a typed construction, not an authority by itself.

UGTS-KC 2.0 added 20 parametric pattern mechanisms, 12 implicit field/surface mechanisms, 10 topology-calculus mechanisms, 12 kinematic mechanisms and 6 bounded dynamics/event mechanisms. Version 3.0 retains them unchanged. Examples include superellipses, the Gielis superformula, Lissajous curves, trochoids, clothoids, Bezier/B-spline/NURBS curves, superquadrics, swept tubes, gyroid and Schwarz-P fields, screw motions, transported frames and event-driven dynamics.

These shapes are not merely display motifs. A curve may be a path, support boundary, route, trajectory template or guard. An implicit field may be a relation, contact gap, phase field or downstream projection. The schema states the role.

8.2 Symmetry and substitution

The 3.0 layer adds finite transform sets for wallpaper and frieze symmetries, Penrose and Ammann-Beenker substitution systems and cut-and-project quasicrystals. Expansion depth and point count are bounded. Quasiperiodicity is a construction, not an assertion of infinite physical resolution.

8.3 Spatial partition and sampling

Voronoi cells, Delaunay adjacency, Lloyd relaxation, Poisson-disk samples and blue-noise diagnostics supply local support and sampling patterns. They may accelerate candidate selection or distribute sensors, but they do not replace compatibility or event certification.

8.4 Indexing and multiresolution

Hilbert and Morton keys provide locality-preserving indices with explicit quantization. Haar wavelets and Laplacian pyramids provide bounded multiresolution transforms. Graph Laplacian modes provide spectral patterns tied to a declared graph.

8.5 Implicit field example

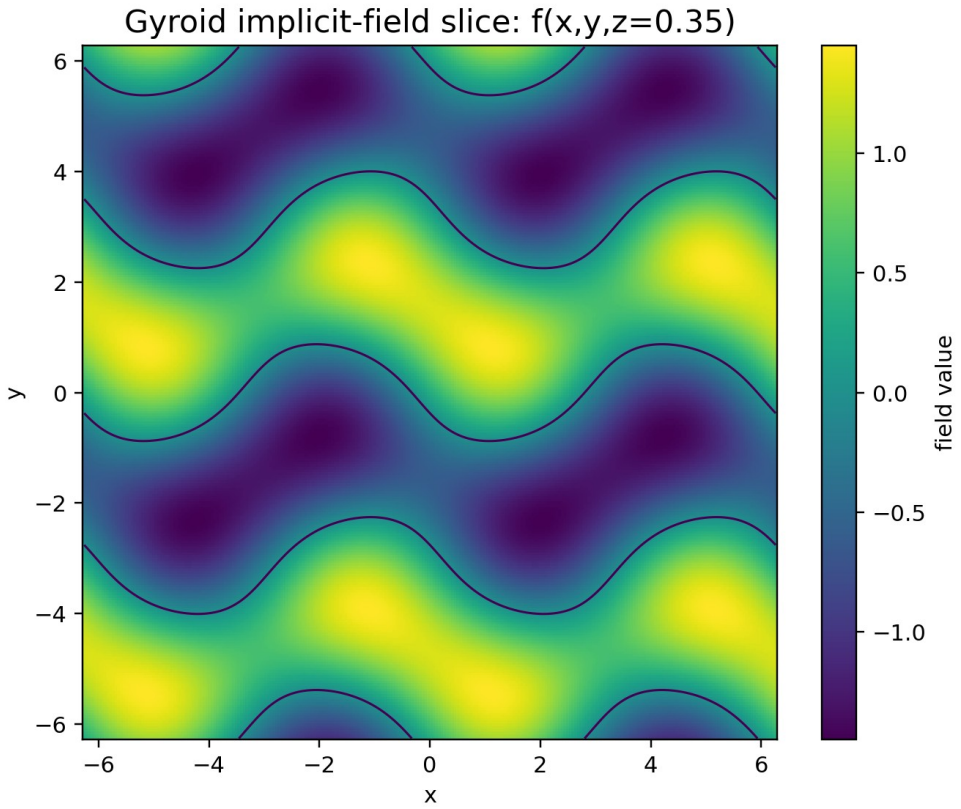


Figure 8. Gyroid scalar-field slice with the zero contour overlaid. Exact-distance status is not implied.

The gyroid slice demonstrates how a periodic scalar field can define supports, phase regions or event surfaces. It is not automatically an exact SDF. Gradients, zero sets and discretization error must be treated separately.

9. Geometric and field dynamics

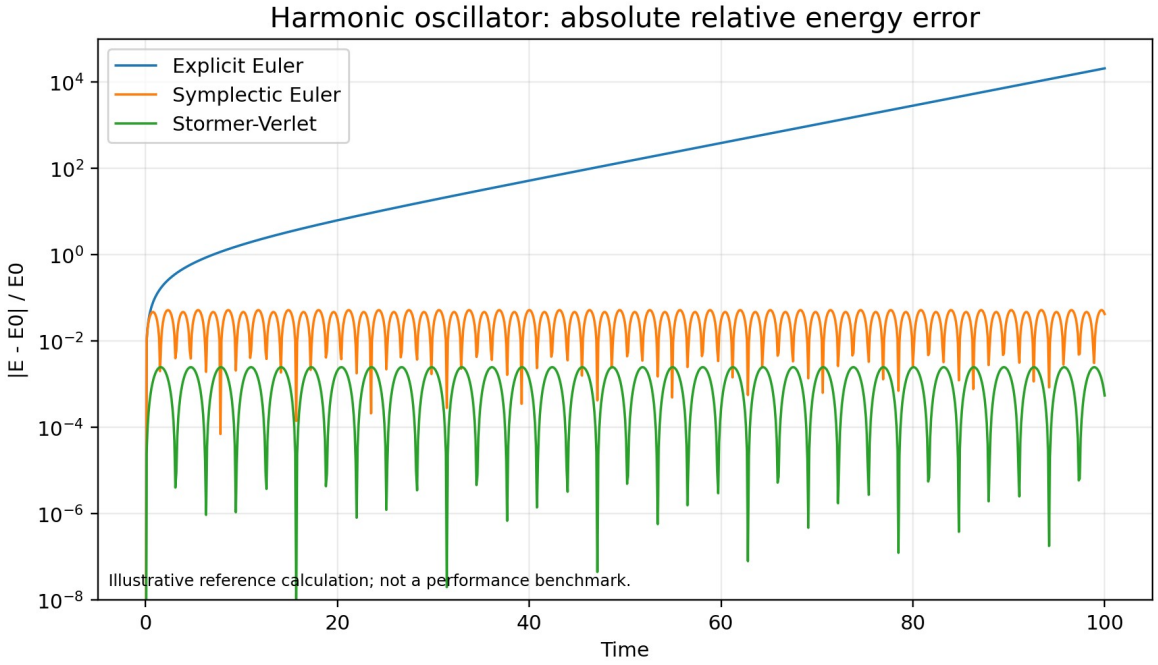


Figure 9. Illustrative harmonic-oscillator energy error. This is a reference calculation, not a performance benchmark.

9.1 Structured mechanical dynamics

The 3.0 package names Hamiltonian canonical flow, Euler-Lagrange flow and discrete variational integration. It implements reference steps for implicit midpoint, symplectic Euler, Stormer-Verlet, velocity Verlet, Lie-group quaternion integration and projected symplectic constraints.

Structure-preserving methods are not universally superior, but they expose invariants and often bound long-time drift better than naive explicit updates. The bundled harmonic-oscillator figure is an illustrative calculation, not a hardware benchmark.

9.2 Field dynamics

The field layer includes:

- semi-Lagrangian advection,
- implicit diffusion,
- wave-equation leapfrog with a CFL check,
- Gray-Scott reaction-diffusion and operator splitting,
- Allen-Cahn phase relaxation,
- Cahn-Hilliard mass-conserving phase evolution,
- graph diffusion,
- eikonal fast-sweeping update.

Every step declares grid spacing, time step, boundary treatment and stability or solver policy. A field update does not become authoritative event state until a relation and verification policy interprets it.

9.3 Event-dynamics coupling

Dynamics proposes a continuous evolution. Event calculus detects the earliest valid guard, advances to the event enclosure, applies the atomic transition, projects constraints if required and restarts the local flow. This avoids stepping through a topology or contact change as if it were ordinary smooth motion.

10. Uncertainty, error budgets and reproducible replay

10.1 Interval and affine uncertainty

The reference package includes closed interval arithmetic, interval vector norms and simple affine forms. Intervals bound possible values under conservative assumptions; affine forms retain selected correlation. Neither is a substitute for a validated physical uncertainty model.

10.2 Statistical propagation

Covariance propagation and unscented sigma points are provided for declared stochastic state. A covariance matrix is kept distinct from a hard interval and from a confidence score. The event record identifies the uncertainty representation used.

10.3 Tolerance policy

A numeric policy contains absolute, relative, time, guard and topology tolerances plus a certification mode. Comparisons use the policy rather than scattered magic constants. Changing tolerance is a schema-visible operation that may change event ordering.

10.4 Deterministic reductions and sampling

A deterministic seed contract, stable pairwise reduction and fixed tie-break rules reduce replay drift. They do not guarantee bitwise equality across every math library or processor. The capability manifest states the reproducibility level.

10.5 Canonical hashes, Merkle chain and checkpoints

Normalized finite JSON can be hashed with sorted keys and schema version. Event hashes are chained so mutation, omission or reordering changes the head. A checkpoint stores world state, schema hash and chain head for bounded rollback.

These are integrity mechanisms. They are not authentication, access control, a cryptographic identity for an entity or proof that the physical input was truthful.

10.6 Error-budget ledger

A verified event may aggregate modeling, discretization, solver, quantization, calibration and measurement allocations. Promotion fails when the total bound exceeds the guard margin or when uncertainty can change event order.

11. World schema, ABI and capability routing

11.1 Schema 3.0

The Draft 2020-12 schema introduces:

- explicit jet fields and frame/unit metadata,
- numeric tolerance and certification policy,
- capability manifest and fallback map,
- relations with support, compatibility, transition, priority, direction and solver,
- holonomic, nonholonomic and unilateral-contact constraints,
- dynamics and integrator records with error budgets,
- topology descriptors,
- simultaneous-event, tie-break, Zeno, atomic-batch and dwell policies.

A validated example world models a planar traveler crossing an `x=0` portal, changing sheet and orientation, carrying a circle constraint and a bounded ground-contact policy.

11.2 Jet and contact ABI

The catalog specifies a structure-of-arrays jet-state ABI and a compact contact-event record. A production binary layout must still declare field widths, alignment, endianness, precision, units, frame identifiers and version. Compact records are accepted only when error stays below the event margin.

11.3 Event compaction

M359 defines a prefix-scan event-compaction contract for parallel backends. The bundled Python fallback is stable CPU compaction. The package does not report a new GPU benchmark for this 3.0 layer.

11.4 Capability manifest

Backends publish supported mechanism IDs, precision, certification level and fallback routes. A world may reject execution when a required mechanism is absent, or it may follow an explicit fallback. Silent degradation is prohibited.

12. Reference implementation and examples

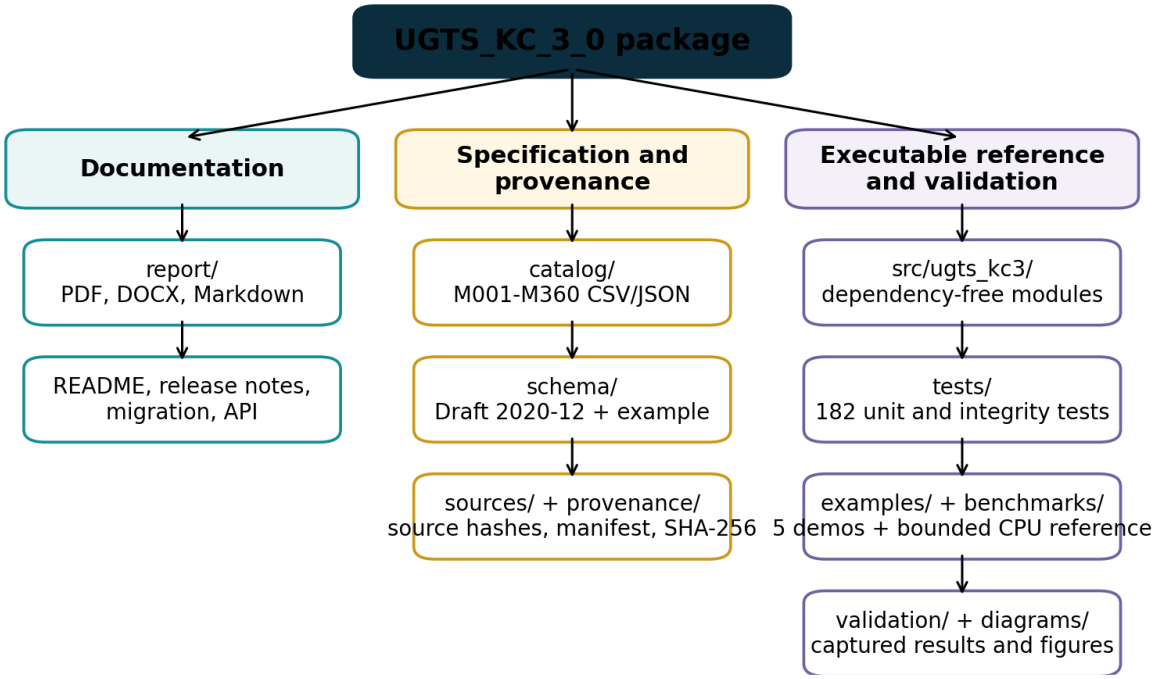


Figure 10. Delivered package layout and verification artifacts.

12.1 Package modules

Module	Responsibility
`kinematics.py`	Jet state, SE(2)/SE(3), quaternions, dual quaternions, moving frames and derivatives.
`constraints.py`	Constraint projection, multipliers, impact, restitution, friction and contact reduction.
`events.py`	Classification, clustering, priority, Zeno detection, interval Newton and Sturm utilities.
`topology.py`	Simplicial structures, persistence, filtrations, Hodge Laplacian, invariants and monodromy.
`patterns.py` / `fields.py`	Parametric patterns, implicit fields and periodic surfaces.
`multiscale.py`	Symmetries, substitutions, spatial partitions, sampling, indices and transforms.
`dynamics.py`	Mechanical and field-dynamics reference steps.
`uncertainty.py`	Intervals, affine forms, covariance, sigma points, hashes, checkpoints and error budgets.
`world.py`	Query-first world for state, compatibility, next event and transition processing.
`io.py`	JSON I/O, minimal validation and capability-manifest helpers.

12.2 Example sequence

```
from ugts_kc3.world import QueryWorld

world = QueryWorld.from_json_file("schema/example_world_v3.json")
state = world.state_at("traveler_A", 1.25)
event = world.next_event("traveler_A", 0.0, 5.0)
record = world.process_next_event("traveler_A", 0.0, 5.0)
```

The examples also demonstrate contact impulses, H0 persistence, field dynamics and uncertainty/replay integrity.

12.3 Package layout

The ZIP contains editable report sources, machine-readable catalogs, schema, code, tests, examples, diagrams, captured validation and provenance manifests. Source PDFs are deliberately excluded.

13. Validation and bounded measurements

13.1 Automated tests

The final test run completed 182 tests:

Test file	Tests	Coverage focus
`test_catalog_integrity.py`	4	Identifier continuity, counts and layer boundaries.
`test_events_topology.py`	39	Event classification, certification, priorities, persistence and topology.
`test_kinematics_constraints.py`	39	Jet/Lie operations, frames, constraints, impact and friction.
`test_multiscale_dynamics.py`	34	Patterns, indices, transforms, mechanical and field dynamics.
`test_patterns_fields.py`	37	2.0 parametric curves, surfaces, SDFs and field algebra.
`test_uncertainty_io_world.py`	29	Intervals, replay integrity, schema-adjacent I/O and query world.

All tests passed. The run establishes internal consistency of the bounded implementation, not external physical performance.

13.2 Schema and examples

The schema is valid Draft 2020-12 JSON Schema. The example world validates against it. All five examples executed and their outputs are captured under `validation/`.

13.3 Reference benchmark boundary

The bundled benchmark is a Python CPU microbenchmark for regression and rough cost visibility. It measures superformula evaluation, SE(3) exponential, interval Newton for `sqrt(2)` and H0 persistence on a 128-vertex case. Results are recorded with Python version, platform, iteration count and checksum.

It is not a GPU, FPGA, ASIC, photonic or universal throughput claim. It must not be compared across machines without preserving software version, workload, precision and measurement method.

14. Limits, kill criteria and roadmap

14.1 Kill criteria

The 3.0 approach should be rejected or simplified for a target workload when any of the following holds:

- support and compatibility fail to remove enough candidates,
- simultaneous-event classes grow faster than the avoided materialization,
- interval widths or quantization exceed the guard margin,
- contact or topology ordering is unstable under the declared tolerance,
- Zeno regularization dominates the model,
- constraint projection or warm-start state destroys deterministic replay,
- grammar or substitution growth escapes its budget,
- topology computations exceed the scale for the bounded reference algorithms,
- event compaction, global coordination or rollback dominates throughput,
- calibration, drift, actuation or sidecar cost erases a physical-device advantage,
- a conventional integrator, collision pipeline or database index is cheaper, faster or more accurate at equal error.

14.2 Recommended 3.x roadmap

1. Freeze the 3.0 exchange schema and mechanism semantics.
2. Add property-based and randomized differential tests while keeping deterministic seeds.
3. Add certified polynomial/interval event support for more restricted surface-trajectory pairs.
4. Implement sparse constraint graphs and a production contact manifold solver.
5. Add scalable persistence through an established topology backend while preserving the exchange contract.
6. Implement and benchmark prefix-scan event compaction on a named physical GPU.
7. Compare query-first event processing against fixed-step and conventional CCD baselines at equal error.
8. Only after software baselines, connect a measured hardware endpoint with full-system energy and calibration.

14.3 Final decision

UGTS-KC 3.0 is a coherent upgrade because it does not abandon the source architecture. It strengthens the layer between geometry and event commitment: motion becomes a typed jet, constraints and contact become explicit relations, difficult roots receive bounded status, simultaneous events commit atomically, topology receives measurable invariants and filtrations, dynamics receive structured integrators, and replay receives integrity and error contracts.

The package is ready for bounded formalization, testing, backend implementation and falsifiable comparison. Projection and physical realization remain downstream and evidence-bound.

Appendix A. M258-M360 mechanism register

These 103 rows are engineering additions introduced in UGTS-KC 3.0. Full records, including layer, disposition and source-boundary fields, are supplied as CSV and JSON.

ID	Domain	Mechanism	Normalized technical definition	Validation
M258	Differential kinematics	Jet-state tower	Augment typed state with position, velocity, acceleration, jerk and snap, each with units and frame metadata.	Implemented + tested
M259	Differential kinematics	Material derivative	Derivative of a field along a trajectory: Df/Dt = partial_t f + grad(f) dot v.	Implemented + tested
M260	Differential geometry	Covariant derivative contract	Differentiate vector fields with an explicit connection or chart metric; ordinary component derivatives are not coordinate invariant.	Specified
M261	Differential geometry	Lie bracket of vector fields	Commutator [X,Y]=DY*X-DX*Y used to expose noncommuting local flows.	Implemented + tested
M262	Lie kinematics	Adjoint frame transfer	Transform twists and wrenches between rigid frames with the SE(3) adjoint.	Implemented + tested
M263	Lie kinematics	SE(3) left Jacobian	Velocity-to-displacement Jacobian with stable small-angle series.	Implemented + tested
M264	Lie kinematics	SE(3) logarithm	Recover a bounded rigid-body twist from a transform under a declared rotation branch.	Prototype + tested
M265	Lie kinematics	Dual-quaternion rigid transform	Represent rotation and translation by a normalized real/dual quaternion pair.	Implemented + tested
M266	Lie kinematics	Dual-quaternion blending	Weighted normalized blend for rigid transforms with hemisphere alignment; not a general deformation theorem.	Implemented + tested
M267	Manifold kinematics	Geodesic interpolation	Interpolate state on a declared manifold using exp/log or a retraction rather than component-wise blending.	Implemented + tested
M268	Manifold kinematics	Retraction update	Map tangent increments back to a manifold when a full exponential is unavailable.	Specified
M269	Differential kinematics	Curvature and torsion invariant	Compute curvature and torsion from first three derivatives with degeneracy status.	Implemented + tested
M270	Frame transport	Bishop-frame holonomy	Accumulate net normal-frame rotation after parallel transport around a closed path.	Prototype + tested
M271	Trajectory timing	Limit-aware time scaling	Compute monotone path timing under velocity, acceleration and jerk budgets.	Prototype + tested
M272	Trajectory synthesis	Differential flatness reconstruction	Recover state and controls from a flat output for a declared model such as a planar unicycle.	Implemented + tested
M273	Constraint dynamics	Holonomic constraint manifold	Equality constraint phi(q,t)=0 with value, Jacobian and tolerance contracts.	Implemented + tested
M274	Constraint dynamics	Nonholonomic Pfaffian constraint	Velocity constraint A(q,t)*qdot=b(q,t) that cannot generally be integrated to position-only form.	Specified
M275	Constraint dynamics	Constraint Jacobian	Derivative J=partial phi/partial q used for projection, multipliers and rank checks.	Implemented + tested
M276	Constraint dynamics	Baumgarte stabilization	Feedback term -2*zeta*omega*phi_dot-omega^2*phi to control drift, with explicit tuning.	Implemented + tested
M277	Constraint dynamics	SHAKE position projection	Iteratively project a trial position onto holonomic constraints within tolerance.	Implemented + tested
M278	Constraint dynamics	RATTLE velocity projection	Project velocity to the tangent space after the position correction.	Implemented + tested
M279	Constraint dynamics	Lagrange multiplier solve	Solve constrained acceleration or impulse through J M^-1 J^T with singularity status.	Implemented + tested
M280	Constraint dynamics	Null-space constrained acceleration	Project unconstrained acceleration into the Jacobian null space.	Implemented + tested
M281	Contact calculus	Unilateral gap function	Signed separation g(q)>=0 distinguishes open, touching and penetrating states.	Implemented + tested
M282	Contact calculus	Complementarity contact condition	Normal impulse lambda>=0, gap velocity/acceleration >=0 and lambda*g=0 under a declared time-stepping model.	Implemented + tested
M283	Contact calculus	Normal impact impulse	Impulse along contact normal computed from effective mass and relative normal velocity.	Implemented + tested
M284	Contact calculus	Coefficient-of-restitution law	Post-impact separating velocity is bounded by a declared restitution coefficient and activation threshold.	Implemented + tested
M285	Contact calculus	Coulomb friction cone	Tangential impulse norm is bounded by mu times normal impulse.	Implemented + tested
M286	Contact calculus	Friction-pyramid approximation	Polyhedral approximation of the friction cone for linear or complementarity solvers.	Implemented + tested

ID	Domain	Mechanism	Normalized technical definition	Validation
M287	Contact calculus	Contact manifold reduction and warm start	Select a bounded representative contact set and seed the next solve from cached impulses.	Prototype + tested
M288	Hybrid event calculus	Guard direction classification	Classify rising, falling, neutral and indeterminate crossings from guard values and derivatives.	Implemented + tested
M289	Hybrid event calculus	Tangency event classification	Distinguish crossing, touch, coincident and unresolved cases using value/derivative tolerances.	Implemented + tested
M290	Hybrid event calculus	Grazing-bifurcation marker	Emit a diagnostic when a guard touches zero with near-zero normal velocity and changes event topology.	Implemented + tested
M291	Hybrid event calculus	Simultaneous event equivalence class	Group event times within a declared temporal enclosure before transition resolution.	Implemented + tested
M292	Hybrid event calculus	Event priority partial order	Resolve dependencies through an acyclic precedence graph, not an undocumented list order.	Implemented + tested
M293	Hybrid event calculus	Deterministic tie-break key	Stable key from time bucket, priority, relation ID and lineage hash.	Implemented + tested
M294	Hybrid event calculus	Zeno accumulation detector	Detect shrinking inter-event intervals approaching a finite accumulation horizon and apply a declared policy.	Implemented + tested
M295	Hybrid event calculus	Hysteresis dwell timer	Require a guard to remain beyond a threshold for a minimum duration before latching.	Implemented + tested
M296	Certified numerics	Event interval enclosure	Represent event time as [t_lo,t_hi] plus residual and direction status.	Implemented + tested
M297	Certified numerics	Lipschitz root exclusion	Exclude a root from an interval when the guard magnitude exceeds a valid Lipschitz displacement bound.	Implemented + tested
M298	Certified numerics	Interval Newton certification	Contract a root interval using interval derivative bounds and classify unique/no/unresolved roots.	Implemented + tested
M299	Certified numerics	Sturm polynomial root count	Count distinct real polynomial roots in an interval without sampling sign changes.	Implemented + tested
M300	Hybrid event calculus	Rollback-safe atomic transition batch	Apply a simultaneous transition set to a snapshot, validate invariants, then commit or roll back as one record.	Implemented + tested
M301	Discrete topology	Oriented simplex incidence	Signed face incidence of oriented simplices, with mod-2 option.	Implemented + tested
M302	Discrete topology	Chain-complex consistency	Verify boundary composed with boundary is zero.	Implemented + tested
M303	Discrete topology	Homology Betti vector	Compute low-dimensional Betti numbers over GF(2) from boundary-matrix ranks.	Implemented + tested
M304	Persistent topology	Union-find H0 persistence	Track connected-component births and merges over an edge filtration.	Implemented + tested
M305	Persistent topology	Persistence barcode interval	Store feature birth, death, dimension, representative and confidence.	Implemented + tested
M306	Persistent topology	Persistence-diagram distance	Bound diagram change with a deterministic greedy matching approximation; exact bottleneck is optional.	Prototype + tested
M307	Persistent topology	Vietoris-Rips filtration	Create vertices, edges and triangles below scale thresholds for bounded point sets.	Implemented + tested
M308	Persistent topology	Alpha-complex proxy	Use Delaunay-admissible simplices and circumradius thresholds; current reference is a bounded 2D proxy.	Prototype + tested
M309	Persistent topology	Cubical lower-star filtration	Order grid cells by maximum incident scalar value and preserve face-before-coface ties.	Implemented + tested
M310	Discrete exterior calculus	Discrete Hodge Laplacian	Assemble combinatorial k-Laplacian from adjacent boundary operators.	Implemented + tested
M311	Discrete topology	Harmonic cycle representative	Represent a persistent cycle by a bounded chain chosen under a declared optimization.	Specified
M312	Topology invariant	Winding and linking invariant	Use winding exactly in 2D and a discretized Gauss linking estimate for separated closed 3D curves.	Prototype + tested
M313	Algebraic topology	Fundamental-group presentation	Store generators and relations for route classes; simplification is bounded and explicit.	Implemented + tested
M314	Covering topology	Covering-space monodromy	Apply loop-induced permutations to sheet labels.	Implemented + tested
M315	Persistent event calculus	Topological change event	Emit an event when a persistence feature crosses a declared lifetime or confidence threshold.	Implemented + tested
M316	Symmetry pattern	Wallpaper-group transform set	Finite generator set for one of the 17 plane symmetry groups, with bounded lattice window.	Prototype + tested
M317	Symmetry pattern	Frieze-group transform set	Finite generator set for strip symmetries over a declared period window.	Prototype + tested
M318	Substitution pattern	Penrose substitution system	Finite-depth thick/thin rhombus substitution with explicit inflation factor and symbol budget.	Prototype + tested
M319	Substitution pattern	Ammann-Beenker substitution system	Finite-depth square/rhombus substitution counts under an explicit matrix.	Prototype + tested

ID	Domain	Mechanism	Normalized technical definition	Validation
M320	Quasiperiodic pattern	Cut-and-project quasicrystal	Select projected lattice points whose internal-space coordinate lies in a bounded acceptance window.	Implemented + tested
M321	Spatial partition	Voronoi cell field	Assign each query to its nearest seed with deterministic tie policy.	Implemented + tested
M322	Spatial partition	Delaunay adjacency	Connect seed pairs sharing an empty-circumcircle simplex in bounded 2D sets.	Implemented + tested
M323	Spatial relaxation	Centroidal Voronoi relaxation	Move seeds toward sample-weighted cell centroids with bounded relaxation.	Implemented + tested
M324	Sampling pattern	Poisson-disk sampling	Deterministic Bridson-style sample generation with minimum-distance and domain contracts.	Implemented + tested
M325	Sampling pattern	Blue-noise spectral criterion	Report nearest-neighbor and low-frequency occupancy diagnostics rather than claiming perfect blue noise.	Implemented + tested
M326	Spatial indexing	Hilbert space-filling index	Map bounded integer grid coordinates to a locality-preserving 2D Hilbert index.	Implemented + tested
M327	Spatial indexing	Morton / Z-order index	Interleave coordinate bits under an explicit width.	Implemented + tested
M328	Multiresolution pattern	Haar wavelet basis	Orthogonal dyadic analysis/synthesis with padding policy.	Implemented + tested
M329	Multiresolution pattern	Laplacian pyramid	Repeated smoothing/downsampling plus residual bands with reconstruction contract.	Implemented + tested
M330	Spectral pattern	Graph Laplacian mode pattern	Use eigenmodes of a symmetric graph Laplacian as bounded spatial patterns.	Prototype + tested
M331	Geometric dynamics	Hamiltonian canonical flow	Advance qdot=partial H/partial p and pdot=-partial H/partial q with a declared integrator.	Implemented + tested
M332	Geometric dynamics	Euler-Lagrange flow	Derive motion from a declared Lagrangian and constraints; symbolic derivation is outside the reference runtime.	Specified
M333	Geometric dynamics	Discrete variational integrator	Stationarity of a discrete action produces an update map with momentum structure.	Prototype + tested
M334	Numerical dynamics	Implicit midpoint method	Second-order implicit update solved to a declared residual tolerance.	Implemented + tested
M335	Geometric dynamics	Stormer-Verlet splitting	Kick-drift-kick splitting for separable Hamiltonians.	Implemented + tested
M336	Lie-group dynamics	Lie-group orientation integrator	Update orientation through group multiplication rather than additive matrix drift.	Implemented + tested
M337	Constraint dynamics	Projected symplectic integrator	Symplectic trial step followed by position and velocity projection, with projection error logged.	Implemented + tested
M338	Field dynamics	Semi-Lagrangian advection	Trace characteristics backward and interpolate under a declared boundary policy.	Implemented + tested
M339	Field dynamics	Implicit diffusion step	Solve the backward-Euler tridiagonal diffusion system for stable large time steps.	Implemented + tested
M340	Field dynamics	Wave-equation leapfrog	Centered second-order time update with CFL contract.	Implemented + tested
M341	Field dynamics	Reaction-diffusion operator splitting	Separate diffusion and reaction updates with declared ordering and substeps.	Implemented + tested
M342	Phase-field dynamics	Allen-Cahn step	Nonconservative phase relaxation with double-well potential and diffusion.	Implemented + tested
M343	Phase-field dynamics	Cahn-Hilliard step	Conservative phase separation through a chemical-potential Laplacian.	Implemented + tested
M344	Arrival-time dynamics	Eikonal fast-sweeping update	Monotone local solution of |grad T|=1/F on a Cartesian grid with positive speed.	Implemented + tested
M345	Certified numerics	Interval scalar arithmetic	Outward-safe interval operations for bounded reference calculations.	Implemented + tested
M346	Certified numerics	Interval vector-norm bound	Lower and upper bounds for Euclidean norm from component intervals.	Implemented + tested
M347	Uncertainty calculus	Affine uncertainty form	First-order correlated uncertainty representation with interval remainder.	Prototype + tested
M348	Uncertainty calculus	Covariance propagation	Linearized covariance update P_next=F P F^T + Q.	Implemented + tested
M349	Uncertainty calculus	Unscented sigma-point propagation	Deterministic nonlinear moment approximation under declared alpha, beta and kappa.	Implemented + tested
M350	Reproducibility	Deterministic Monte Carlo seed contract	Key pseudo-random streams by schema, lineage, mechanism and sample index.	Implemented + tested
M351	Numerical policy	Floating-point tolerance policy	Centralize absolute, relative, time, guard and topology tolerances in the world schema.	Implemented + tested
M352	Reproducibility	Deterministic reduction tree	Pairwise reduction with a fixed tree independent of thread scheduling.	Implemented + tested
M353	Reproducibility	Canonical serialization hash	Hash normalized JSON with sorted keys, finite-number checks and schema version.	Implemented + tested

ID	Domain	Mechanism	Normalized technical definition	Validation
M354	Reproducibility	Event-log Merkle chain	Chain event hashes to detect omission, reordering or mutation.	Implemented + tested
M355	Reproducibility	Replay checkpoint	Snapshot state, schema hash and event-chain head for bounded rollback/replay.	Implemented + tested
M356	Verification	Error-budget ledger	Allocate and aggregate modeling, discretization, quantization, solver and measurement error.	Implemented + tested
M357	Runtime ABI	Structure-of-arrays jet-state ABI	Separate position, velocity, acceleration, topology and uncertainty streams with versioned strides.	Specified + schema
M358	Runtime ABI	Compact contact-event record	Bounded record carrying time interval, contact IDs, normal, impulse, status and lineage hash.	Specified + schema
M359	GPU runtime	Prefix-scan event compaction contract	Predicate, exclusive scan and stable scatter of verified events with overflow accounting.	Specified
M360	Runtime governance	Capability manifest and fallback routing	Declare supported mechanisms, precision, certification level and deterministic fallback path.	Implemented + tested

Appendix B. Package and source verification

B.1 Source hashes

The supplied PDFs are not included in the ZIP. Their SHA-256 hashes are recorded in `sources/source_pdf_hashes.txt`.

B.2 Package manifest

`provenance/manifest.json` records every payload file's relative path, size and SHA-256 hash. The manifest, checksum list and file list use conventional self-exclusion rules documented inside the manifest. `provenance/SHA256SUMS.txt` hashes the manifest and all payload files except itself.

B.3 Reproduction commands

```
python -m unittest discover -s tests -v
PYTHONPATH=src python examples/query_first_contact_portal.py
PYTHONPATH=src python examples/persistence_demo.py
PYTHONPATH=src python examples/contact_demo.py
PYTHONPATH=src python examples/field_dynamics_demo.py
PYTHONPATH=src python examples/uncertainty_replay_demo.py
PYTHONPATH=src python benchmarks/reference_benchmark.py
```

B.4 Canonical shorthand

```
local support -> compatibility -> certified guard
-> simultaneous event class -> atomic transition
-> route/topology/contact/jet update
-> lineage + novelty + integrity checkpoint
```

End of report.